

Kurdish Medical Dataset Generation with Ollama

Research Implementation

May 1, 2026

Contents

1	Original Request and System Information (prompt.md)	2
2	Installation Scripts	3
2.1	install_ollama.sh	3
2.2	setup_ollama_model.sh	5
2.3	install_requirements.sh	7
3	Server Management	8
3.1	start_ollama_server_first.sh	8
3.2	run_server_bg_first.py	10
4	Dataset Generation	12
4.1	generate_dataset_first.py	12
4.2	run_generator_bg_first.py	16
5	Testing and Master Script	18
5.1	test_ollama.sh	18
5.2	run.sh	19
5.3	pkill_ollama.sh	20

1 Original Request and System Information (prompt.md)

Table 1: Structured summary of `prompt.md`

Item	Details
Original Request	
Requested change	change this code from llamacpp to ollama for installation and running start the server and even file name
Testing constraint	pleases don't run any code for test in my machine because i copy this file to my server just write the code and i will run it in my server this information about my server in below
Model note	also i already downloaded gemma 3 27 b
Operating System & Kernel	
System	Linux
Node Name	a214-Z790-D
Release	5.15.0-139-generic
Version	#149~20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025
Machine	x86_64
Processor	x86_64
CPU Information	
Physical cores	14
Total cores	20
Max Frequency	4620.00 MHz
Current Frequency	3257.71 MHz
CPU Usage	4.4%
GPU Information	
GPU 0	NVIDIA GeForce RTX 4090 D, Load: 0.0%, Free Mem: 48123.0 MB, Used Mem: 405.0 MB, Total Mem: 49140.0 MB, Temperature: 32.0 °C
GPU 1	NVIDIA GeForce RTX 4090 D, Load: 0.0%, Free Mem: 48511.0 MB, Used Mem: 25.0 MB, Total Mem: 49140.0 MB, Temperature: 29.0 °C
Memory Information	
Total	15.44 GB
Available	11.40 GB
Used	4.04 GB
Percentage	26.2%

2 Installation Scripts

2.1 install_ollama.sh

```
1 #!/bin/bash
2 # install_ollama.sh - Install Ollama on Ubuntu (CUDA/GPU enabled)
3 set -e
4
5 echo "===== "
6 echo "  Ollama Installation Script"
7 echo "  Target: Ubuntu 20.04 | Dual RTX 4090 D"
8 echo "===== "
9
10 # 1. Install CURL if not present
11 echo "[*] Ensuring curl is installed..."
12 sudo apt-get update -y
13 sudo apt-get install -y curl
14
15 # 2. Remove any existing Ollama installation before reinstalling
16 echo ""
17 echo "[*] Checking for existing Ollama installation..."
18 OLLAMA_BIN=$(command -v ollama || echo "/usr/local/bin/ollama")
19
20 if [ -f "$OLLAMA_BIN" ] || systemctl list-unit-files | grep -q ollama.service;
21 then
22     echo "[!] Ollama installation detected - removing it completely for a clean
23     reinstall..."
24
25     # Stop and disable the service
26     sudo systemctl stop ollama 2>/dev/null || true
27     sudo systemctl disable ollama 2>/dev/null || true
28
29     # Remove the binary
30     if [ -f "$OLLAMA_BIN" ]; then
31         sudo rm -f "$OLLAMA_BIN"
32         echo "    [v] Binary removed ($OLLAMA_BIN)"
33     fi
34
35     # Also check /usr/bin just in case
36     sudo rm -f /usr/bin/ollama
37
38     # Remove the systemd unit and any drop-in overrides
39     sudo rm -f /etc/systemd/system/ollama.service
40     sudo rm -rf /etc/systemd/system/ollama.service.d
41     sudo systemctl daemon-reload
42     echo "    [v] Systemd unit and overrides removed"
43
44     # Remove system data directory (libraries/runners)
45     if [ -d /usr/share/ollama ]; then
46         sudo rm -rf /usr/share/ollama
47         echo "    [v] System data directory /usr/share/ollama removed"
48     fi
49
50     # Remove the ollama system user/group
51     if id -u ollama &>/dev/null; then
52         sudo userdel ollama 2>/dev/null || true
53         echo "    [v] System user 'ollama' removed"
54     fi
55
56     if getent group ollama &>/dev/null; then
```

```

53     sudo groupdel ollama 2>/dev/null || true
54     echo "        [v] System group 'ollama' removed"
55 fi
56
57     echo "[+] Old Ollama installation removed."
58 else
59     echo "[+] No existing Ollama installation found - proceeding with fresh
install."
60 fi
61 echo ""
62
63 # 3. Fresh Ollama install (automatically detects CUDA and installs GPU support)
64 echo "[*] Running official Ollama installer..."
65 curl -fsSL https://ollama.com/install.sh | sh
66
67 # 3. Configure Ollama storage on the large drive (/home is full)
68 echo "[*] Creating Ollama storage directory on /mnt/storage1/shko/ollama ..."
69 export OLLAMA_HOME="/mnt/storage1/shko/ollama"
70 sudo mkdir -p "$OLLAMA_HOME"
71 # Make the directory owned by the user that will run ollama
72 sudo chown -R "$USER":"$USER" "$OLLAMA_HOME"
73
74 # 4. Configure Ollama to listen on all interfaces (so the Python client can reach
it)
75 echo "[*] Configuring Ollama systemd service (host, GPUs, storage)..."
76 sudo mkdir -p /etc/systemd/system/ollama.service.d
77 cat <<EOF | sudo tee /etc/systemd/system/ollama.service.d/override.conf
78 [Service]
79 Environment="OLLAMA_HOST=0.0.0.0:11434"
80 # Use both GPUs
81 Environment="CUDA_VISIBLE_DEVICES=0,1"
82 # Store all Ollama data (models, blobs, keys) on the large drive
83 Environment="OLLAMA_HOME=/mnt/storage1/shko/ollama"
84 EOF
85
86 # 5. Reload systemd and start the Ollama service
87 echo "[*] Enabling and starting Ollama service..."
88 sudo systemctl daemon-reload
89 sudo systemctl enable ollama
90 sudo systemctl restart ollama
91
92 echo ""
93 echo "===== "
94 echo "    SUCCESS! Ollama is installed and running."
95 echo "    Service status: sudo systemctl status ollama"
96 echo "    API endpoint   : http://localhost:11434"
97 echo "===== "
98 echo ""
99 echo "    NEXT STEPS:"
100 echo "    1. Run ./setup_ollama_model.sh    <- registers Gemma 3 27B"
101 echo "    2. Run python3 run_server_bg_first.py <- starts the generator"
102 echo "===== "

```

Listing 1: install_ollama.sh

2.2 setup_ollama_model.sh

```
1 #!/bin/bash
2 # setup_ollama_model.sh - Register the local Gemma 3 27B GGUF with Ollama
3 # Run this AFTER install_ollama.sh
4 #
5 # The model weight file is expected at:
6 #   model_weights/gemma-3-27b-it-q4_k_m.gguf
7 # (relative to THIS script's directory, i.e. the directory you uploaded to the
   server)
8 #
9 # Ollama's "create" command reads a Modelfile, imports the GGUF blob into its
10 # internal content-addressed store, and registers it as "gemma3:27b".
11 # After this runs, "ollama run gemma3:27b" works offline - no internet needed.
12
13 set -e
14
15 # -- Storage -----
16 export OLLAMA_HOME="/mnt/storage1/shko/ollama"
17 export OLLAMA_MODELS="/mnt/storage1/shko/ollama/Models"
18 mkdir -p "$OLLAMA_HOME"
19 mkdir -p "$OLLAMA_MODELS"
20
21 # -- Paths -----
22 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
23 GGUF_FILE="$SCRIPT_DIR/model_weights/gemma-3-27b-it-q4_k_m.gguf"
24 MODEL_NAME="gemma3:27b"
25 MODELFILE="$SCRIPT_DIR/Modelfile"
26
27 echo "===== "
28 echo "   Ollama Model Setup (local GGUF import)"
29 echo "   Model   : $MODEL_NAME"
30 echo "   GGUF    : $GGUF_FILE"
31 echo "   Home    : $OLLAMA_HOME"
32 echo "   Models  : $OLLAMA_MODELS"
33 echo "===== "
34
35 # -- Sanity checks -----
36 if [ ! -f "$GGUF_FILE" ]; then
37     echo ""
38     echo "[!] ERROR: GGUF file not found at:"
39     echo "    $GGUF_FILE"
40     echo ""
41     echo "    Make sure you transferred the file with the correct path:"
42     echo "    model_weights/gemma-3-27b-it-q4_k_m.gguf"
43     exit 1
44 fi
45
46 if ! curl -s http://localhost:11434/api/tags > /dev/null 2>&1; then
47     echo "[!] Ollama service does not seem to be running."
48     echo "    Start it with:  sudo systemctl start ollama"
49     echo "    Or manually   :  OLLAMA_HOME=$OLLAMA_HOME ollama serve"
50     exit 1
51 fi
52
53 # -- Write a minimal Modelfile -----
54 echo "[*] Writing Modelfile..."
55 cat > "$MODELFILE" <<EOF
```

```

56 # Modelfile for Gemma 3 27B Instruct (Q4_K_M GGUF)
57 FROM $GGUF_FILE
58
59 # Recommended chat template for Gemma 3 instruct models
60 TEMPLATE ""<start_of_turn>user
61 {{ .Prompt }}<end_of_turn>
62 <start_of_turn>model
63 ""
64
65 # Generation parameters
66 PARAMETER temperature 0.7
67 PARAMETER top_p 0.9
68 PARAMETER top_k 40
69 PARAMETER repeat_penalty 1.1
70 PARAMETER num_ctx 8192
71 EOF
72
73 echo "    Modelfile written to: $MODELFILE"
74
75 # -- Import the model -----
76 echo ""
77 echo "[*] Running: ollama create $MODEL_NAME -f $MODELFILE"
78 echo "    This copies the GGUF into Ollama's blob store - may take a minute..."
79 echo ""
80
81 OLLAMA_HOME="$OLLAMA_HOME" OLLAMA_MODELS="$OLLAMA_MODELS" ollama create "
    $MODEL_NAME" -f "$MODELFILE"
82
83 echo ""
84 echo "===== "
85 echo "    SUCCESS! Model is ready."
86 echo ""
87 echo "    Quick test:"
88 echo "        OLLAMA_HOME=$OLLAMA_HOME OLLAMA_MODELS=$OLLAMA_MODELS ollama run
    $MODEL_NAME \"Hello!\""
89 echo ""
90 echo "    List registered models:"
91 echo "        OLLAMA_HOME=$OLLAMA_HOME OLLAMA_MODELS=$OLLAMA_MODELS ollama list"
92 echo ""
93 echo "    The model is now registered as: $MODEL_NAME"
94 echo "    Start the server with: python3 run_server_bg_first.py"
95 echo "===== "

```

Listing 2: setup_ollama_model.sh

2.3 install_requirements.sh

```
1 #!/bin/bash
2 # install_requirements.sh - Python requirements for dataset generation
3 # Ollama runs as its own service, so we only need the OpenAI-compatible client +
   helpers.
4
5 set -e
6
7 echo "Updating system packages..."
8 sudo apt-get update
9
10 echo "Installing system dependencies..."
11 # psmisc provides 'fuser', used in the background manager scripts
12 sudo apt-get install -y psmisc python3-pip python3-venv
13
14 echo "Installing Python requirements for the project..."
15 # openai - speaks to Ollama's /v1 OpenAI-compatible endpoint
16 # httpx - used for fine-grained connection control in the generator
17 pip3 install openai httpx
18
19 echo "-----"
20 echo "Project requirements installed successfully!"
21 echo ""
22 echo "NOTE: Ollama itself is the model server."
23 echo "      No llama-cpp-python, huggingface_hub, or"
24 echo "      GGUF management libraries are needed."
25 echo "-----"
```

Listing 3: install_requirements.sh

3 Server Management

3.1 start_ollama_server_first.sh

```
1 #!/bin/bash
2 # start_ollama_server_first.sh
3 # Ensures the Ollama daemon is running and the Gemma 3 27B model is loaded.
4 # Ollama itself is the server - this script just makes sure it is up.
5
6 # Bypass system proxy for localhost connections
7 export no_proxy="localhost,127.0.0.1,0.0.0.0"
8 export NO_PROXY="localhost,127.0.0.1,0.0.0.0"
9
10 # -- Storage: redirect all Ollama data to the large drive -----
11 # /home is full; /mnt/storage1/shko/ollama has the free space.
12 # OLLAMA_HOME controls where Ollama stores its config and ed25519 key.
13 # OLLAMA_MODELS controls where model blobs (the actual weights) are stored.
14 export OLLAMA_HOME="/mnt/storage1/shko/ollama"
15 export OLLAMA_MODELS="/mnt/storage1/shko/ollama/Models"
16 mkdir -p "$OLLAMA_HOME"
17 mkdir -p "$OLLAMA_MODELS"
18
19 # Context window: allow up to 32 K tokens for both input and output
20 export OLLAMA_NUM_CTX=32768
21
22 # Configuration
23 MODEL_NAME="gemma3:27b"
24 OLLAMA_HOST="0.0.0.0"
25 OLLAMA_PORT=11434
26
27 echo "-----"
28 echo "  Starting / checking Ollama server"
29 echo "  Model : $MODEL_NAME"
30 echo "  Host  : $OLLAMA_HOST"
31 echo "  Port  : $OLLAMA_PORT"
32 echo "  GPUs  : Both RTX 4090 D (CUDA_VISIBLE_DEVICES=0,1)"
33 echo "-----"
34
35 # Export so the Ollama process uses both GPUs
36 export CUDA_VISIBLE_DEVICES=0,1
37 export OLLAMA_HOST="${OLLAMA_HOST}:${OLLAMA_PORT}"
38
39 # If Ollama is running as a systemd service, restart it to pick up env vars
40 if systemctl is-active --quiet ollama 2>/dev/null; then
41     echo "[*] Ollama systemd service is active - restarting to apply env vars..."
42     # Patch the systemd override so the service also knows about OLLAMA_HOME
43     sudo mkdir -p /etc/systemd/system/ollama.service.d
44     sudo tee /etc/systemd/system/ollama.service.d/storage.conf > /dev/null <<EOF
45 [Service]
46 Environment="OLLAMA_HOME=/mnt/storage1/shko/ollama"
47 Environment="OLLAMA_MODELS=/mnt/storage1/shko/ollama/Models"
48 EOF
49     sudo systemctl daemon-reload
50     sudo systemctl restart ollama
51     sleep 3
52 else
53     # Otherwise start it as a foreground process (useful when running manually)
54     echo "[*] Starting Ollama serve in the foreground..."
```



```

55     exec ollama serve
56 fi
57
58 # Wait until the REST API is reachable
59 echo "[*] Waiting for Ollama API to become available..."
60 for i in $(seq 1 30); do
61     if curl -s http://localhost:${OLLAMA_PORT}/api/tags > /dev/null 2>&1; then
62         echo "[+] Ollama API is up!"
63         break
64     fi
65     echo "        ... attempt $i/30"
66     sleep 2
67 done
68
69 # Pre-load the model so the first inference call is instant
70 echo "[*] Pre-loading model $MODEL_NAME into GPU VRAM..."
71 ollama run "$MODEL_NAME" "" 2>/dev/null || true
72
73 echo ""
74 echo "-----"
75 echo "    Ollama is ready."
76 echo "    API endpoint : http://localhost:${OLLAMA_PORT}"
77 echo "    OpenAI compat: http://localhost:${OLLAMA_PORT}/v1"
78 echo "    Model       : $MODEL_NAME"
79 echo "-----"

```

Listing 4: start_ollama_server_first.sh

3.2 run_server_bg_first.py

```
1 import subprocess
2 import os
3 import time
4 import sys
5 from pathlib import Path
6
7 # --- Configuration ---
8 BASE_DIR = Path(__file__).parent.resolve()
9 SERVER_SCRIPT = BASE_DIR / "start_ollama_server_first.sh"
10 SERVER_LOG = BASE_DIR / "ollama_server_logs_first.txt"
11
12 def start_process(command, log_file_handle, description, cwd=None):
13     """Starts a process in the background and redirects output to a log file."""
14     print(f"[*] Starting {description}...")
15
16     process = subprocess.Popen(
17         command,
18         stdout=log_file_handle,
19         stderr=subprocess.STDOUT,
20         preexec_fn=os.setpgrp if sys.platform != "win32" else None,
21         cwd=str(cwd) if cwd else str(BASE_DIR)
22     )
23
24     print(f"[+] {description} started with PID: {process.pid}")
25     return process
26
27 def main():
28     if not SERVER_SCRIPT.exists():
29         print(f"ERROR: {SERVER_SCRIPT} not found.")
30         sys.exit(1)
31
32     # --- Cleanup: stop any stale ollama processes that are NOT the systemd
33     # service ---
34     print(f"[*] Cleaning up any stale non-systemd ollama serve processes...")
35     os.system("pkill -9 -f 'ollama serve' > /dev/null 2>&1 || true")
36
37     print(f"[*] Waiting 3 seconds for resources to be released...")
38     time.sleep(3)
39
40     os.chmod(SERVER_SCRIPT, 0o755)
41
42     print(f"{'='*60}")
43     print(f" Ollama Server Background Manager")
44     print(f"{'='*60}")
45     print(f" Server Log : {SERVER_LOG}")
46     print(f"{'='*60}")
47
48     with open(SERVER_LOG, "a") as server_handle:
49         server_handle.write(f"\n\n{'='*80}\n")
50         server_handle.write(f" OLLAMA SERVER SESSION STARTED AT: {time.strftime('%Y-%m-%d %H:%M:%S')}\n")
51         server_handle.write(f"{'='*80}\n\n")
52         server_handle.flush()
53
54     server_proc = start_process(["bash", str(SERVER_SCRIPT)], server_handle, "Ollama Server", cwd=BASE_DIR)
```

```

54
55     print(f"\n{'='*60}")
56     print(" SUCCESS: Ollama server script is running in the background.")
57     print(f" Server PID : {server_proc.pid}")
58     print(f" Check logs : tail -f {SERVER_LOG}")
59     print(f" API           : http://localhost:11434")
60     print(f" OpenAI API  : http://localhost:11434/v1")
61     print(f"{'='*60}\n")
62     print(f" To stop, use: kill {server_proc.pid}")
63     print(f" or:           sudo systemctl stop ollama")
64
65 if __name__ == "__main__":
66     try:
67         main()
68     except Exception as e:
69         print(f"CRITICAL ERROR: {e}")
70         sys.exit(1)

```

Listing 5: run_server_bg_first.py

4 Dataset Generation

4.1 generate_dataset_first.py

```
1 #!/usr/bin/env python3
2 """
3 generate_dataset_first.py (Ollama version) - First Dataset Instance
4
5 For every .txt file under the configured DATA_DIR:
6 - Standalone files (e.g. 1-10.txt): run the prompt NUM_RUNS times -> JSON
  files
7                                     -> merge all runs into one final JSON
  file
8 - Part files (e.g. 163267_part1.txt, 163267_part2.txt, ...):
9   run the prompt NUM_RUNS times per part -> JSON files per
  part
10                                     -> after ALL parts are done, merge everything into one
  final JSON
11
12 Original .txt files are NEVER deleted.
13 Duplicate (question, response) pairs within the same final merged file are removed
14 .
15 Requires: Ollama running on localhost:11434 with gemma3:27b loaded.
16 """
17
18 import os
19 import re
20 import json
21 import time
22 import httpx
23 from openai import OpenAI
24
25 # ----- Bypass Proxy -----
26 # Ensure localhost connections are NOT routed through any system proxy
27 os.environ["no_proxy"] = "localhost,127.0.0.1"
28 os.environ["NO_PROXY"] = "localhost,127.0.0.1"
29
30 # ----- Configuration -----
31
32 # Use absolute paths for input and output
33 DATA_DIR = "UKH-AIIC-KA_Just_TXT_file/part1"
34 OUTPUT_DIR = "UKH-AIIC-KA_Just_TXT_file_Output/part1"
35 MODEL_NAME = "gemma3:27b"
36 NUM_RUNS = 1
37
38 # Ollama OpenAI-compatible endpoint
39 client = OpenAI(
40     base_url="http://localhost:11434/v1",
41     api_key="ollama", # Ollama ignores the key but the field must be non-
  empty
42     http_client=httpx.Client(proxy=None),
43 )
44
45 # ----- Prompt Template -----
46
47 def build_prompt(text: str, run_index: int) -> str:
48     return f"""Act as an expert Kurdish linguist and data engineer.
```

```

49 Analyze the following text and determine its category (e.g., History, Medical,
    Mathematics, Technology, Literature, Science, etc.).
50 Then, generate high-quality, information-rich question-and-answer pairs based on
    the text.
51
52 IMPORTANT: Return a VALID JSON array. Start your response with '[' and end with
    ']',.
53 Do NOT include any markdown formatting, code fences (''json), or introductory/
    concluding text.
54
55 Structure:
56 [
57     {{
58         "id": "1",
59         "category": "<category>",
60         "question": "<formal Sorani question>",
61         "response": "<formal Sorani answer>",
62         "document": {{
63             "title": "<title>",
64             "source_url": "",
65             "publication_date": ""
66         }}
67     }}
68 ]
69
70 Rules:
71 - Language: Central Kurdish (Sorani) ONLY.
72 - Grammar: Formal and academic level.
73 - Format: Output ONLY the raw JSON array.
74
75 Source text:
76 \"\"\"
77 {text}
78 \"\"\"
79
80
81 # ----- Helpers -----
82
83 def call_model(prompt: str) -> str:
84     retry_wait = 30
85     while True:
86         try:
87             response = client.chat.completions.create(
88                 model=MODEL_NAME,
89                 messages=[{"role": "user", "content": prompt}],
90                 temperature=0.7,
91                 max_tokens=32768,
92             )
93             return response.choices[0].message.content
94         except (httpx.ConnectError, httpx.ConnectTimeout, httpx.
RemoteProtocolError) as e:
95             print(f"\n    x Server connection lost: {e}")
96             time.sleep(retry_wait)
97         except Exception as e:
98             print(f"\n    x Error calling model: {e}")
99             time.sleep(retry_wait)
100
101 def extract_json_array(raw: str) -> list:
102     if not raw or not isinstance(raw, str):

```

```

103         return []
104     cleaned = re.sub(r"```(?:json)?\s*", "", raw)
105     cleaned = cleaned.replace("```", "").strip()
106     start = cleaned.find("[")
107     end = cleaned.rfind("]")
108     if start != -1 and end != -1 and end > start:
109         json_str = cleaned[start : end + 1]
110         json_str = re.sub(r",\s*([\]}])", r"\1", json_str)
111         try:
112             data = json.loads(json_str)
113             if isinstance(data, list): return data
114             if isinstance(data, dict): return [data]
115         except json.JSONDecodeError: pass
116     return []
117
118 def deduplicate(records: list) -> list:
119     seen = set()
120     unique = []
121     for rec in records:
122         key = (rec.get("question", "").strip(), rec.get("response", "").strip())
123         if key not in seen and key != ("", ""):
124             seen.add(key)
125             unique.append(rec)
126     return unique
127
128 def re_index(records: list, offset: int = 0) -> list:
129     for i, rec in enumerate(records, start=offset + 1):
130         rec["id"] = str(i)
131     return records
132
133 def save_json(path: str, data: list):
134     with open(path, "w", encoding="utf-8") as f:
135         json.dump(data, f, ensure_ascii=False, indent=2)
136     print(f"    [SAVE] Saved {len(data)} records -> {os.path.relpath(path)}")
137
138 def load_json(path: str) -> list:
139     with open(path, "r", encoding="utf-8") as f:
140         return json.load(f)
141
142 def process_single_txt(txt_path: str, output_dir: str, base_name: str):
143     print(f"\n    [FILE] Processing: {os.path.relpath(txt_path)}")
144     with open(txt_path, "r", encoding="utf-8") as f:
145         text = f.read()
146     run_files = []
147     for run in range(1, NUM_RUNS + 1):
148         run_output = os.path.join(output_dir, f"{base_name}_run{run}.json")
149         if os.path.exists(run_output):
150             try:
151                 existing = load_json(run_output)
152                 if len(existing) > 0:
153                     run_files.append(run_output)
154                 continue
155             except: pass
156         prompt = build_prompt(text, run)
157         raw = call_model(prompt)
158         records = extract_json_array(raw)
159         if records:
160             save_json(run_output, records)
161             run_files.append(run_output)

```

```

162         time.sleep(2)
163     all_records = []
164     for rf in run_files:
165         if os.path.exists(rf):
166             try: all_records.extend(load_json(rf))
167             except: pass
168     return re_index(deduplicate(all_records))
169
170 def process_folder(input_folder: str, output_folder: str):
171     os.makedirs(output_folder, exist_ok=True)
172     all_files = sorted(f for f in os.listdir(input_folder) if f.endswith(".txt"))
173     standalone = [f for f in all_files if "_part" not in f or f.startswith("part_")
174 ]
175     parts = [f for f in all_files if "_part" in f and not f.startswith("part_")]
176     for fname in standalone:
177         txt_path = os.path.join(input_folder, fname)
178         base_name = os.path.splitext(fname)[0]
179         final_out = os.path.join(output_folder, f"{base_name}_FINAL.json")
180         if os.path.exists(final_out): continue
181         merged = process_single_txt(txt_path, output_folder, base_name)
182         if merged: save_json(final_out, merged)
183     if parts:
184         groups = {}
185         for fname in parts:
186             group_key = re.sub(r"_part\d+\.txt$", "", fname)
187             groups.setdefault(group_key, []).append(fname)
188         for group_key, part_files in groups.items():
189             final_out = os.path.join(output_folder, f"{group_key}_FINAL.json")
190             if os.path.exists(final_out): continue
191             group_records = []
192             for fname in sorted(part_files):
193                 txt_path = os.path.join(input_folder, fname)
194                 base_name = os.path.splitext(fname)[0]
195                 merged = process_single_txt(txt_path, output_folder, base_name)
196                 group_records.extend(merged)
197             if group_records: save_json(final_out, re_index(deduplicate(
198 group_records)))
199
200 def main():
201     if not os.path.exists(DATA_DIR): return
202     process_folder(DATA_DIR, OUTPUT_DIR)
203
204 if __name__ == "__main__":
205     main()

```

Listing 6: generate_dataset_first.py

4.2 run_generator_bg_first.py

```
1 import subprocess
2 import os
3 import time
4 import sys
5 from pathlib import Path
6
7 # --- Configuration ---
8 BASE_DIR = Path(__file__).parent.resolve()
9 GENERATOR_SCRIPT = BASE_DIR / "generate_dataset_first.py"
10 GENERATOR_LOG = BASE_DIR / "generate_dataset_logs_first.txt"
11 PYTHON_EXEC = sys.executable
12
13 def start_process(command, log_file_handle, description, cwd=None):
14     """Starts a process in the background and redirects output to a log file."""
15     print(f"[*] Starting {description}...")
16
17     process = subprocess.Popen(
18         command,
19         stdout=log_file_handle,
20         stderr=subprocess.STDOUT,
21         preexec_fn=os.setpgrp if sys.platform != "win32" else None,
22         cwd=str(cwd) if cwd else str(BASE_DIR)
23     )
24
25     print(f"[+] {description} started with PID: {process.pid}")
26     return process
27
28 def main():
29     if not GENERATOR_SCRIPT.exists():
30         print(f"ERROR: {GENERATOR_SCRIPT} not found.")
31         sys.exit(1)
32
33     # --- Selective Cleanup ---
34     my_pid = os.getpid()
35     print(f"[*] Cleaning up old generator processes for {GENERATOR_SCRIPT.name}...")
36     os.system(f"pgrep -f '{GENERATOR_SCRIPT.name}' | grep -v '^{my_pid}$' | xargs
37 -r kill -9 2>/dev/null || true")
38     time.sleep(2)
39
40     print(f"{'='*60}")
41     print(f" Dataset Generator Background Manager")
42     print(f"{'='*60}")
43     print(f" Generator Log : {GENERATOR_LOG}")
44     print(f"{'='*60}")
45
46     with open(GENERATOR_LOG, "a") as generator_handle:
47         generator_handle.write(f"\n\n{'='*80}\n")
48         generator_handle.write(f" DATASET GENERATOR SESSION STARTED AT: {time.
49 strftime('%Y-%m-%d %H:%M:%S')}\n")
50         generator_handle.write(f"{'='*80}\n\n")
51         generator_handle.flush()
52
53     generator_proc = start_process([PYTHON_EXEC, str(GENERATOR_SCRIPT)],
54 generator_handle, "Dataset Generator", cwd=BASE_DIR)
```



```

53     print(f"\n{' '*60}")
54     print(" SUCCESS: Generator is running in the background.")
55     print(f" Generator PID : {generator_proc.pid}")
56     print(f" Check logs      : tail -f {GENERATOR_LOG}")
57     print(f"{' '*60}\n")
58     print(f" To stop, use: kill {generator_proc.pid}")
59
60 if __name__ == "__main__":
61     try:
62         main()
63     except Exception as e:
64         print(f"CRITICAL ERROR: {e}")
65         sys.exit(1)

```

Listing 7: run_generator_bg_first.py

5 Testing and Master Script

5.1 test_ollama.sh

```
1 #!/bin/bash
2 # test_ollama.sh
3 # Tests whether the Ollama server and gemma3:27b model are working correctly.
4
5 # -- Config --
6 MODEL_NAME="gemma3:27b"
7 OLLAMA_PORT=11434
8 BASE_URL="http://localhost:${OLLAMA_PORT}"
9
10 echo "===== "
11 echo "  Ollama Test Suite"
12 echo "===== "
13
14 # -- Test 1: API reachability --
15 HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" "${BASE_URL}/api/tags")
16 if [ "$HTTP_CODE" = "200" ]; then
17     echo "[PASS] Ollama API is reachable"
18 else
19     echo "[FAIL] Ollama API is NOT reachable"
20     exit 1
21 fi
22
23 # -- Test 2: Model is listed --
24 if curl -s "${BASE_URL}/api/tags" | grep -q "${MODEL_NAME}"; then
25     echo "[PASS] Model '${MODEL_NAME}' found"
26 else
27     echo "[FAIL] Model '${MODEL_NAME}' NOT found"
28     exit 1
29 fi
30
31 # -- Test 3: Inference --
32 RESPONSE=$(curl -s -X POST "${BASE_URL}/api/generate" \
33     -H "Content-Type: application/json" \
34     -d "{
35         \"model\": \"${MODEL_NAME}\",
36         \"prompt\": \"Reply with confirm you are working.\",
37         \"stream\": false
38     }")
39
40 if echo "$RESPONSE" | grep -q "response"; then
41     echo "[PASS] Inference OK"
42 else
43     echo "[FAIL] Inference failed"
44 fi
```

Listing 8: test_ollama.sh

5.2 run.sh

```
1 # 1. Install Ollama + GPU support
2 chmod +x install_ollama.sh && ./install_ollama.sh
3
4 # 2. Register / download the Gemma 3 27B model
5 chmod +x setup_ollama_model.sh && ./setup_ollama_model.sh
6
7 # 3. Install Python dependencies
8 chmod +x install_requirements.sh && ./install_requirements.sh
9
10 # 4. Start the Ollama server in background
11 python3 run_server_bg_first.py
12
13 # 5. Start the dataset generator in background
14 python3 run_generator_bg_first.py
15
16 # To stop everything:
17 chmod +x pkill_ollama.sh && ./pkill_ollama.sh
```

Listing 9: run.sh

5.3 pkill_ollama.sh

```
1 #!/bin/bash
2 # pkill_ollama.sh - Stop all Ollama and dataset-generator processes
3
4 echo[STOP]Terminating all Ollama and generator processes..."
5
6 # Stop the systemd Ollama service (if running)
7 if systemctl is-active --quiet ollama 2>/dev/null; then
8     echo "    [*] Stopping ollama systemd service..."
9     sudo systemctl stop ollama
10 fi
11
12 # Kill any manually started ollama serve processes
13 pkill -9 -f "ollama serve"      2>/dev/null || true
14 pkill -9 -f "run_server_bg"     2>/dev/null || true
15 pkill -9 -f "run_generator_bg"  2>/dev/null || true
16 pkill -9 -f "generate_dataset"  2>/dev/null || true
17
18 # Free the Ollama default port just in case
19 fuser -k 11434/tcp 2>/dev/null || true
20
21 echo [OK]All killed."
```

Listing 10: pkill_ollama.sh